MIDI to Score with Pretrained MusicXML Decoder

Yi-Chieh Chiu ychiu443@usc.edu
Bo Gao gaobo@usc.edu
Hao Jiang hjiang28@usc.edu
Siyuan Bao siyuanba@usc.edu
Ioannis Tsotras itsotras@usc.edu

Abstract

MIDI-to-score conversion aims to translate symbolic performance data into clean, human-readable musical notation. Although MIDI preserves pitch and timing information, it lacks many score-level structures required for readable notation, especially under expressive timing noise. In this project, we formulate noisy MIDI-to-score conversion as a sequence-to-sequence task and study whether MusicXML decoder pre-training improves downstream score generation. Our system uses a CPWord-based MIDI encoder and a Linearized MusicXML decoder within an asymmetric Transformer architecture. The decoder is pretrained on bar-aware MusicXML sequences and then adapted to noisy MIDI-to-score fine-tuning with curriculum learning. Experiments show that curriculum learning improves convergence, full fine-tuning outperforms LoRA in our setting, and decoder pre-training improves downstream validation performance over training from scratch. These results suggest that MusicXML decoder pre-training is a promising direction for improving the robustness and structural consistency of neural MIDI-to-score systems.

1 Introduction

Automatic Music Transcription (AMT) aims to automatically convert acoustic music recordings into human-readable musical scores. It can be divided into two steps: converting audio to an intermediate representation (like MIDI), and translating that discrete format into a human-readable musical score. Our research focuses on this latter, highly complex MIDI-to-Score (MIDI2Score) sub-task.

While MIDI precisely captures physical note properties, it inherently lacks the semantic structure necessary for readability. This conversion is challenging due to expressive timing noise, micro-fluctuations, and the ambiguity of notation. Because determining optimal notation requires a deep understanding of musical semantics, traditional rule-based algorithms frequently fail.

Consequently, researchers increasingly frame MIDI2Score as a sequence-to-sequence (Seq2Seq) translation problem using Transformer models. However, unlike modern Natural Language Processing (NLP), which relies on large-scale pre-training to enhance contextual understanding, existing MIDI2Score models are primarily trained from scratch.

Recent releases of large-scale datasets (e.g., MetaScore, PDMX) and optimized tokenization strategies (e.g. CPWord, Linearized MusicXML) now provide the infrastructure to bridge this gap. Therefore, this project explores the impact of large-scale decoder pre-training on the MIDI2Score task.

2 Related Work

Performance MIDI-to-score conversion transforms expressive performance recordings into structured, human-readable notation. Recent work increasingly formulates this as an end-to-end sequence modeling task. Beyer and Dai (Beyer and Dai, 2024) propose a Transformer-based framework that jointly performs note quantization and score engraving from expressive piano MIDI. Liu et al. (Liu et al., 2022) decompose the task into beat tracking and score reconstruction, aligning expressive timing before symbolic conversion. Murgul and Heizmann (Murgul and Heizmann, 2025) further improve robustness through stronger beat and downbeat tracking models. However, these methods either tightly couple rhythm quantization with engraving or depend on task-specific intermediate supervision. In contrast, we directly treat noisy MIDI-to-score conversion as a unified encoder-decoder translation problem.

For clean symbolic input, the task is closer to score generation or engraving. Score Transformer (Suzuki, 2022) uses a Transformer to trans-

late note-level symbolic sequences into notation-level tokens while jointly predicting score attributes such as staff assignment, voice separation, and beams. EngravingGNN (Karystinaios et al., 2025) instead formulates engraving as a graph-based multi-task prediction problem over note relationships. These methods achieve strong engraving quality but assume clean symbolic input and are typically trained from scratch. Our work bridges this gap by explicitly modeling noisy symbolic inputs and introducing large-scale decoder pre-training to learn structural priors over valid MusicXML-style sequences.

Recent work has also explored unified frameworks for music translation across modalities. Jung et al. (Jung et al., 2025) propose a unified encoder-decoder Transformer for translating between score images, symbolic notation, MIDI, and audio. While effective for multimodal translation, these approaches prioritize modality alignment over task-specific optimization for MIDI-to-score generation. Our work adopts a similar sequence-to-sequence formulation, but focuses specifically on MIDI2Score and uses an asymmetric architecture in which the encoder denoises noisy MIDI representations while the pretrained decoder generates structurally consistent score tokens.

3 Problem Description

We study the MIDI-to-Score problem: given a symbolic performance representation, the goal is to generate a clean, human-readable score that is both musically correct and engraving-consistent. In our setting, the input is a CPWord-tokenized MIDI sequence and the output is a linearized MusicXML (LMX) sequence. Formally, given an input sequence $x = (x_1, \ldots, x_m)$ and a target sequence $y = (y_1, \ldots, y_n)$, the model learns the conditional distribution $p(y \mid x)$.

This problem is challenging because the mapping from performance-like MIDI to notation is not one-to-one. MIDI preserves note events and timing, but does not explicitly encode many score-level decisions needed for readable notation, such as voice allocation, beaming, and staff assignment. Noise and expressive timing variation further obscure the intended score structure. As a result, the model must perform both *denoising* and *notation reconstruction* while still generating syntactically valid MusicXML sequences.

We formulate this task as sequence-to-sequence generation with an asymmetric Transformer: the encoder interprets noisy CPWord-based MIDI inputs, while the decoder generates LMX tokens. Within this formulation, our main question is whether decoder pre-training on large-scale MusicXML-style sequences can provide useful structural priors for downstream MIDI-to-score generation.

4 Methods

4.1 Tokenization

For MusicXML, we use Linearized MusicXML (LMX) token (Mayer et al., 2024) and the linearized-musicxml library for tokenization. After transforming the dataset into LMX tokens, we also train BPE tokenizer on the training dataset with huggingface tokenizer library. We also apply BPE dropout (Provilkov et al., 2020) with dropout rate of 0.1 to enhance model robustness.

For the MIDI part, we choose CPWord (Hsiao et al., 2021) for tokenization and the implementation from the miditok library with some custom configurations.

4.2 Data

We use the PDMX dataset as source of MusicXML data. According to their paper (Long et al., 2025), the $R \cap D$ subset with higher quality data would produce better result, so we extract this subset for our experiment.

For training Seq2Seq model, we need dataset with paired MusicXML and MIDI data. To obtain such data, we adopt the method used by (Suzuki, 2022), transforming the MusicXML to MIDI (with music21 library) to form pairs.

Since our task is to convert noisy MIDI data to score, we need to inject noise into perfectly accurate MIDI data. To mimic the nuances of human performance, we applied three types of noise: onset jitter, onset drift, and duration jitter, simulating touch error, speed variation, and duration instability, respectively. We generate three different intensities of noise: clean (without noise), light, and heavy, with the parameters described in Table 1. The target noise ratios for model training are 2:3:5, so that the model can learn to handle different levels of noise.

After tokenizing the data, considering that simple segmentation and truncation would disrupt the musical structure and the alignment between the score and MIDI, we applied a bar-line based seg-

mentation/truncation method to the data in advance. We only truncate the musical content at bar lines to ensure the score remains intact and valid. Furthermore, the truncation points for both MIDI and MusicXML tokens are located on the same bar line, ensuring consistency between the two types of content.

Before truncation, the entire dataset is divided into training, validating, and testing sets in a 16:4:5 ratio based on the total length of the LMX tokens. For simplicity, we break down each score into parts according to the instruments, so that each piece of data is a single part. During the splitting process, we ensure that different parts of the same source score are within the same split to avoid data leakage and to make the sequence length distribution similar across different splits.

4.3 Model Architecture

Inspired by the deep-shallow architecture proposed by (Kasai et al., 2020), we use an asymmetric encoder-decoder architecture with an 8-layer encoder and a 4-layer decoder for efficiency. The encoder is expected to denoise the MIDI inputs and obtain proper representation, and the decoder is expected to generate grammatically correct LMX outputs.

For both encoder and decoder, we uses hidden size 512, 8 attention heads, and feed-forward dimension 2048. We use RMSNorm in a pre-norm residual layout (Zhang and Sennrich, 2019), tie the input embedding matrix and output projection matrix of the decoder (Press and Wolf, 2017), set the maximum training context length to 1024 tokens. The final configuration uses rotary positional embeddings (RoPE) (Su et al., 2021) and GELU as the feed-forward activation (Hendrycks and Gimpel, 2016). These choices, together with the final chunking and regularization settings, were selected through controlled ablations over positional encoding, overlap strategy, dropout, and activation on decoder pre-trining.

Since CPWord encodes MIDI signals as a two-dimensional array (with multiple token with different type at each timestep), we must design our own embedding layer to generate the embedding vectors that will be input into the encoder. For each token type at each timestep, we use independent embedding layers, and then project the concatenated embedding to the target size (512 in our case) with a linear layer. The configuration detail is presented in Table 2.

4.4 Decoder Pre-Training

We first train a decoder-only language model on symbolic music sequences before downstream sequence-to-sequence fine-tuning. This stage is motivated by the observation that MIDI-to-score systems are often trained from scratch, even though large symbolic music corpora, recent symbolic-music pre-training results, and improved symbolic tokenization pipelines now make large-scale decoder pre-training feasible (Sheng et al., 2021; Qu et al., 2024; Fradet et al., 2023). Given a tokenized linearized MusicXML sequence $x = (x_1, \ldots, x_T)$, we optimize the standard next-token prediction objective $p(x_t \mid x_{<t})$ over all valid target positions. The resulting decoder is then used to initialize the downstream MIDI-to-score model. Model architecture details are described separately; here we focus on the pre-training objective, sequence construction, batching strategy, and optimization recipe.

A key component of the pipeline is bar-aware sequence construction. Instead of slicing tokenized music into fixed windows, we build training chunks bar by bar using explicit `measure` boundaries, following recent symbolic-music work that emphasizes bar-level structure (Qu et al., 2024; Chen et al., 2024). We set the chunk limit to 1024 BPE tokens and reuse the last two bars of the previous chunk as context for the next one (`overlap_bars = 2`). Beginning-of-sequence and end-of-sequence tokens are added only to the first and last chunks of each piece, respectively. The resulting chunks are then converted into input IDs through the tokenizer pipeline described in Section 4.1.

Because bar-aware chunking produces variable-length examples, we use length bucketing together with dynamic batching (Ott et al., 2019). Examples with similar lengths are grouped before batching, and each batch is constrained by a fixed token budget rather than a fixed example count. In the final setup, we use `max_tokens_per_batch = 16384` and pad sequence lengths to a multiple of 64.

We optimize the decoder with AdamW (Loshchilov and Hutter, 2019) using learning rate 6×10^{-4}, weight decay 0.1, $\beta_1 = 0.9$, and $\beta_2 = 0.95$. We apply gradient clipping with maximum norm 2.0 and use cosine learning rate decay (Loshchilov and Hutter, 2017) with 500 warmup steps and a minimum learning rate ratio of 0.1. The final recipe uses dropout 0.1, validation every 500 steps, and early stopping with patience 10 and minimum improvement threshold

0.001, under a maximum budget of 24 epochs. The final promoted configuration combines RoPE, GELU, bar-aware chunking with `overlap_bars = 2`, and length-bucketed dynamic batching with `max_tokens_per_batch = 16384`. Under this configuration, the best pre-training run achieved validation loss 1.56, corresponding to perplexity 4.76. Table 3 shows the 24-epoch ablation set used to choose the final design.

To select the final recipe, we ran controlled 24-epoch ablations over positional encoding, dropout, and activation while keeping the rest of the pipeline fixed. RoPE (Su et al., 2021) improved over sinusoidal positional encoding (Vaswani et al., 2017), dropout 0.1 improved over 0.05, and GELU (Hendrycks and Gimpel, 2016) further improved over SwiGLU (Shazeer, 2020). These comparisons determined the final pre-training configuration used in the rest of the project.

4.5 Seq2Seq Fine-Tuning

In the Seq2Seq training phase, we load the pre-trained decoder weights into the decoder model and fine-tune them on the Noisy MIDI-to-Score task.

Unlike decoder pre-training, which aims to familiarize the model with grammatical rules, this phase requires the model to generate consistent content. Since much important musical information often only appears at the beginning of the score (including clefs, key signatures, and time signatures), we do not use chunking but directly truncate excessively long sequences.

Similar to decoder pre-training, we also use length bucketing to improve training efficiency. However, because we also need to consider the length of CPWord we add the expected CPWord sequence length to the LMX length in a weighted manner (multiplied by 0.2) for sorting.

Considering that learning a large amount of heavy noise from the beginning might be too difficult for the model, we apply curriculum learning. In the early stages of training, we only provide clean (no noise) data to allow the model to quickly learn the meaning of CPWord tokens, gradually increasing the proportion of noise, and finally using the target noise level ratio for training. In actual training, we divided the training into three stages with different noise probability distributions, switching between them according to the training epoch. Details are shown in Table 4.

For model fine-tuning, we considered two approaches: Full Fine-tuning and LoRA. In Full Fine-tuning, we set the decoder parameter learning rate to 0.1 times the encoder learning rate to avoid catastrophic forgetting. In LoRA fine-tuning, we froze the decoder parameters and injected LoRA layers into all linear layers (including attention qkv projection and FFN layers) with LoRA rank of 16, alpha of 32, and a dropout of 0.05.

The training configuration is similar to our decoder pre-trainning. We optimize our model with AdamW using the learning rate 3×10^{-4}, the weight decay 0.1, $\beta_1 = 0.9$, and $\beta_2 = 0.999$. The learning rate is scheduled with cosine learning rate decay with a minimum learning rate ratio of 0.1. For LoRA fine-tuning, the warmup step is set as 66 (2% of 16 epochs), but for full fine-tuning, to prevent catastrophic forgetting, the warmup step is set as 328 (10% of 16 epochs). The dropout rate is set to 0.1 and the gradient clipping value is set to 1.0.

4.6 Evaluation Metrics

Our evaluation is performed on a normalized event-based representation rather than direct XML string matching. Specifically, we use `partitura` (Cancino-Chacón et al., 2022) to parse both predicted and reference MusicXML files into standard score objects, and then convert them into unified note events with pitch, pitch class, onset, duration, staff, and voice information to make the evaluation less sensitive to surface-level XML variation while preserving the symbolic structure required for score comparison.

To keep the evaluation concise but informative, we focus on four representative metrics showing in Table 5. **Music Content Score** serves as the main content-oriented metric, summarizing pitch-class correctness, rhythmic correctness, note-level temporal alignment, and global timeline similarity. It is intended to reflect whether the predicted score is musically close to the reference. **Notation Score** evaluates score readability and structural faithfulness by combining staff-aware note matching, measure-level correctness, score attributes, and XML structural similarity.

To complement these two composite scores, we additionally report two interpretable component metrics. **Pitch-Class-Onset-Duration F1** measures whether the correct pitch classes appear at approximately the correct onset times and durations, and therefore captures fine-grained note-event alignment. **Rhythm F1** removes pitch from

the matching condition and evaluates only onset and duration consistency.

For threshold-based note matching, we use an onset tolerance of 0.25 beats and a duration tolerance of 0.25 beats. In addition to overall results, we also report grouped performance on the clean, light, and heavy subsets to analyze robustness under different input noise levels.

4.7 Inference

After training the seq2seq model, we load the final checkpoint for inference. We implemented a key-value cache mechanism to reduce inference costs. Considering that we found the model exhibited hallucinations in our initial experiments, generating highly repetitive content, we added a repetition penalty with a value of 1.2 to mitigate it.

5 Experiment Results

In order to find the best way to do the seq2seq fine-tuning, we conduct four experiments with 16 epochs of training for each, and use the best configuration to train 32 epochs to obtain the main result. The validation is run at the end of each epoch. All the following experiments ran on NVIDIA A40.

5.1 Experiment 1: Curriculum Learning

The purpose of this experiment is to examine the effectiveness of curriculum learning training strategies: whether curriculum learning assists model learning and improves learning efficiency, or whether reducing the time spent learning the target noise level negatively impacts final performance.

We use the LoRA fine-tuning method for this experiment, comparing the loss curves with and without curriculum learning, keeping other parameters constant. In training without curriculum learning, we trained based on the target noise level distribution from the beginning. The results are shown in Figure 1.

As can be seen, curriculum learning not only helps overcome the initial chaos in fewer training steps, but also results in a lower final loss, meeting our expectations. This shows that curriculum learning contributes to convergence and achieves better performance.

5.2 Experiment 2: LoRA vs. Full Fine-Tuning

The purpose of this experiment is to compare the training performance differences between LoRA and Full Fine-Tuning, including training time, peak GPU usage, and loss curve. The results are shown in Figure 1.

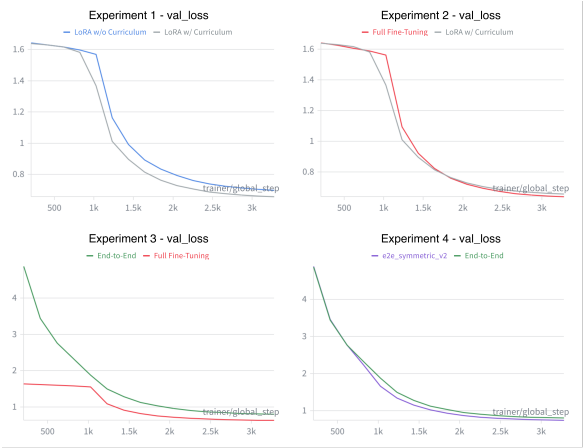

Figure 1: Validation Loss Curves for Experiments

The results show that while the Full Fine-Tuning version experienced longer initial confusion, it ultimately achieved a lower loss than the LoRA version. The reason behind it may be that in the early stages of training, the chaotic signals from the encoder slightly disrupted the decoder that was not frozen, but later the decoder had more room to adjust and better align with the new MIDI-to-Score task.

From a training cost perspective, LoRA fine-tuning took 115 minutes with a peak GPU memory allocation of 37.6GB, while full fine-tuning took 107 minutes and 39.6GB. This shows that the cost of Full fine-tuning was not significantly different from LoRA. This can be explained by the fact that the frozen parameters was not the majority in our training: the encoder had twice as many layers, and the decoder's cross-attention was not frozen. Thus, LoRA's efficiency was not high for our purpose comparing to full fine-tuning.

5.3 Experiment 3: Decoder Pre-Training

The purpose of this set of experiments is to verify whether our approach of pre-training the decoder is effective. Our method is to use an decoder with randomly initialized weights for our seq2seq model, trained with almost the same parameter settings as the full fine-tuning model, and compare the loss curve with the one with full fine-tuning. The difference is that, because we don't need to worry about catastrophic forgetting in end-to-end training, we did not set a discriminative learning rate, and the warmup step was set to 2% of the total steps instead of 10%.

Ideally, we should keep the total training amount of both to be consistent in order to compare which one has higher training efficiency. However, since the resources consumed by training the decoder alone are difficult to directly correspond to the training of seq2seq, and the training of decoder and seq2seq are carried out on different machines, it is difficult to compare the training time. Therefore, we unify the number of seq2seq training epochs for both to 16, and try to prove that pre-training is beneficial as a prior knowledge initialization. The results are shown in Figure 1.

The results show that full fine-tuning outperforms end-to-end methods, proving the hypothesis that pre-training is beneficial. This is likely because end-to-end models need to learn both LMX grammar and CPWord representation and denoising simultaneously whereas the pretrained decoder already captures part of the target-side structure. However, this difference is not decisive. In terms of token accuracy, the final difference is only 4% (83.15% vs. 79.63%).This suggests that encoder learning remains a major factor, and that stronger end-to-end performance may still be achievable with additional training.

5.4 Experiment 4: Asymmetric Architecture

The purpose of this set of experiments is to evaluate the impact of our asymmetric architecture design on performance. The method involves training a classic 6-6 layer transformer architecture and comparing it with our asymmetric model. Since we did not have time to pre-train a decoder with a different number of layers, we only compared models trained using the end-to-end method. Except for the number of layers, the parameters of the symmetric architecture model were consistent with the previous end-to-end training stage. The results are shown in Figure 1.

The results showed that although the symmetric architecture did have a lower loss than the asymmetric architecture, the difference was very small, with only a 1% difference in final token accuracy (80.99% vs. 79.62%). Considering the benefit of fewer decoder layers to inference speed, we believe this trade-off is worthwhile.

5.5 Main Result and Comparison

After the above experiments, we choose to use the best configuration, which is full fine-tuning with curriculum learning, to train more steps (32 epochs) to obtain our final main result.

We evaluate our approach against three baselines: music21, MuseScore, and the Beyer model proposed in recent work (Beyer and Dai, 2024). music21 is an open-source Python library for symbolic music processing and analysis, commonly used for programmatic conversion between MIDI and MusicXML. MuseScore is a professional music notation software designed for high-quality score engraving and typesetting, and provides a strong baseline for evaluating visually and structurally coherent musical output. The Beyer model is also a recent learning-based approach for MIDI-to-score translation. All results are shown in Figure 2.

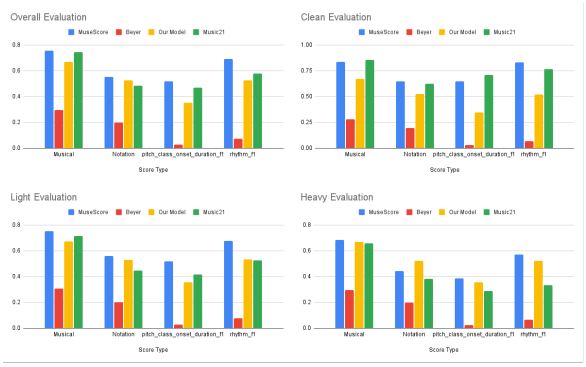

Figure 2: Performance comparison across all models

Across all settings (overall, clean, light, and heavy), our model consistently performs competitively against music21, MuseScore, and the Beyer model. The Beyer model remains the weakest baseline across most metrics, likely because its data representation, training distribution, and output formulation are less aligned with our noisy MIDI-to-score setting, leading to weaker note-level and rhythmic reconstruction. In contrast, the rule-based systems do not hallucinate nonexistent measures or large-scale score structures as easily, which helps them achieve relatively strong results on clean inputs but degrades more noticeably as the input becomes noisier. In contrast, our model maintains stable performance across noise conditions, achieving consistently strong results, particularly under light and heavy perturbations.

6 Conclusion

In this study, we trained an asymmetric encoder-decoder architecture model to handle noisy MIDI-to-score tasks. We improved the model's performance using decoder pre-training and curriculum learning, and experimentally verified the effectiveness of these methods. While our proposed model

ultimately lost to rule-based tools on low-noise data, it demonstrated greater robustness on high-noise data. Although our model is still some distance from practical application in music production workflows, it still demonstrates the potential of model-based methods and provides some guidance for future development.

7 Future Work

On the tokenizer side, since CPWord is designed for generation task originally, it can be more optimized to our task.

On the pre-training side, a natural next step is to scale the decoder language model further in model size, training budget, and corpus coverage. It is also worth exploring stronger structure-aware pre-training schemes, such as longer-context modeling or phrase-level sequence construction, to better capture musical form beyond the current bar-aware setup.

On the application side, our current max context length is too short for real world usage, but it is possible to apply RoPE scaling techniques to make it applicable for longer sequences. Also, LMX and CPWord is actually capable of processing multi-track music, so if the context length is longer, it can be used for processing full score directly.

On the evaluation side, our current framework still mostly tied to absolute pitch and absolute timing. As a result, predictions with uniform temporal shift may be penalized more than their musical similarity would suggest. A natural next step is to introduce more relative and alignment-aware metrics. It is also worth exploring higher-level measures to better capture long-range musical structure beyond local note matching.

8 Individual Contributions

8.1 Yi-Chieh Chiu

Led the project. Designed and implemented the entire data pipeline, including pre-processing, tokenization and bar-aware truncation (excluding the decoder chunking part). Implemented advanced features of seq2seq model and training procedure based on the framework built by Hao Jiang, including curriculum learning, length bucketing of seq2seq, full fine-tuning setup and more. Designed all the experiments and executed them on CARC.

8.2 Bo Gao

Led the decoder pre-training component of the project. Implemented the pre-training pipeline, including the decoder language model architecture, bar-aware chunk construction, length-bucketed dynamic batching, and the overall training and model-selection process; ran the main pre-training ablations over chunk overlap, positional encoding, dropout, and activation; analyzed validation results to select the final pre-training recipe; and prepared the final pretrained checkpoint used for downstream fine-tuning.

8.3 Hao Jiang

Led the seq2seq fine-tuning and inference implementation of the project. Implemented the sequence-to-sequence training framework and adapted the model architecture for MIDI-to-score translation. Integrated the CPWord-based encoder input processing and embedding mechanism into the training pipeline. Also implemented the inference and generation pipeline for decoding MusicXML-style outputs from MIDI inputs and producing final predictions for downstream evaluation.

8.4 Siyuan Bao

Led the evaluation and result-analysis component of the project. Implemented the partitura-based MusicXML evaluation pipeline, including note-level, rhythm-level, measure-level, and structure-aware metrics for comparing predicted and reference scores. Designed and summarized the results with composite content and notation scores. Adjusted and extended the experiment scripts and evaluation output interfaces to support batch result generation, grouped reporting, and external baseline comparison.

8.5 Ioannis Tsotras

Led the MIDI-to-MusicXML preprocessing and conversion pipeline for the baseline evaluation. Implemented an automated workflow to extract MIDI files from the dataset and convert them into structured MusicXML format using both music21 and MuseScore. Measured baseline statistics on the converted data to assess structural differences introduced by varying levels of noise and to establish reference points for downstream evaluation.

References

Tim Beyer and Angela Dai. 2024. End-to-end Piano Performance-MIDI to Score Conversion with Transformers. *arXiv preprint*. ArXiv:2410.00210 [cs].

Carlos Cancino-Chacón, Maarten Grachten, and Thassilo Gadermaier. 2022. Partitura: A python package for symbolic music processing. *arXiv preprint arXiv:2206.01071*.

Haonan Chen, Jordan B. L. Smith, Janne Spijkervet, Ju-Chiang Wang, Pei Zou, Bochen Li, Qiuqiang Kong, and Xingjian Du. 2024. SymPAC: Scalable symbolic music generation with prompts and constraints. *arXiv preprint arXiv:2409.03055*.

Nathan Fradet, Nicolas Gutowski, Fabien Chhel, and Jean-Pierre Briot. 2023. Byte pair encoding for symbolic music. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1993–2010. Association for Computational Linguistics.

Dan Hendrycks and Kevin Gimpel. 2016. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*.

Wen-Yi Hsiao, Jen-Yu Liu, Yin-Cheng Yeh, and Yi-Hsuan Yang. 2021. Compound Word Transformer: Learning to Compose Full-Song Music over Dynamic Directed Hypergraphs. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(1):178–186. Number: 1.

Jongmin Jung, Dongmin Kim, Sihun Lee, Seola Cho, Hyungjoon So, Irmak Bukey, Chris Donahue, and Dasaem Jeong. 2025. U-must: A unified framework for cross-modal translation of score images, symbolic music, and performance audio. *IEEE Transactions on Audio, Speech and Language Processing*.

Emmanouil Karystinaios, Francesco Foscarin, and Gerhard Widmer. 2025. EngravingGNN: A Hybrid Graph Neural Network for End-to-End Piano Score Engraving. *arXiv preprint*. ArXiv:2509.19412 [cs].

Jungo Kasai, Nikolaos Pappas, Hao Peng, James Cross, and Noah A Smith. 2020. Deep encoder, shallow decoder: Reevaluating non-autoregressive machine translation. *arXiv preprint arXiv:2006.10369*.

Lele Liu, Qiuqiang Kong, GV Morfi, Emmanouil Benetos, and 1 others. 2022. Performance midi-to-score conversion by neural beat tracking.

Phillip Long, Zachary Novack, Taylor Berg-Kirkpatrick, and Julian McAuley. 2025. PDMX: A Large-Scale Public Domain MusicXML Dataset for Symbolic Music Processing. In *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5.

Ilya Loshchilov and Frank Hutter. 2017. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*.

Jiří Mayer, Milan Straka, Jan Hajič, and Pavel Pecina. 2024. Practical End-to-End Optical Music Recognition for Pianoform Music. In *Document Analysis and Recognition - ICDAR 2024*, pages 55–73, Cham. Springer Nature Switzerland.

Sebastian Murgul and Michael Heizmann. 2025. Beat and Downbeat Tracking in Performance MIDI Using an End-to-End Transformer Architecture. *arXiv preprint*. ArXiv:2507.00466 [cs].

Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, and Michael Auli. 2019. fairseq: A fast, extensible toolkit for sequence modeling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*, pages 48–53. Association for Computational Linguistics.

Ofir Press and Lior Wolf. 2017. Using the output embedding to improve language models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*, pages 157–163. Association for Computational Linguistics.

Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. 2020. Bpe-dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1882–1892.

Xingwei Qu, Yuelin Bai, Yinghao Ma, Ziya Zhou, Ka Man Lo, Jiaheng Liu, Ruibin Yuan, Lejun Min, Xueling Liu, Tianyu Zhang, Xinrun Du, Shuyue Guo, Yiming Liang, Yizhi Li, Shangda Wu, Junting Zhou, Tianyu Zheng, Ziyang Ma, Fengze Han, and 10 others. 2024. MuPT: A generative symbolic music pretrained transformer. *arXiv preprint arXiv:2404.06393*.

Noam Shazeer. 2020. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*.

Zhonghao Sheng, Kaitao Song, Xu Tan, Yi Ren, Wei Ye, Shikun Zhang, and Tao Qin. 2021. MusicBERT: Symbolic music understanding with large-scale pretraining. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 791–800. Association for Computational Linguistics.

Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*.

Masahiro Suzuki. 2022. Score Transformer: Generating Musical Score from Note-level Representation. In *Proceedings of the 3rd ACM International Conference on Multimedia in Asia*, MMAsia '21, pages 1–7, New York, NY, USA. Association for Computing Machinery.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*.

Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32.

9 Appendix

	Light	Heavy
Onset Jitter	$\lambda \sim N(0,$ 10ms)	$\lambda \sim N(0, 25\text{ms})$
Onset Drift	$\lambda \sim U$(-16th, 16th)	$\lambda \sim U(-8th, 8th)$
Duration Jitter	$t' = t \cdot U(0.9, 1.1)$	$t' = t \cdot U(0.8, 1.2)$

Table 1: N and U are normal and uniform distribution. $16th$ and $8th$ is 16th and 8th note in music.

Token Type	Vocab Size	Embedding Size
Family	5	8
Position	133	64
Pitch	132	64
Duration	104	64
Program	133	32
Tempo	36	32
Time Sig.	68	32

Table 2: The configuration of the embedding layer for CPWord inputs. The vocabulary of each type include special tokens [PAD], [BOS], [EOS].

Ablation	Setting	Best val loss
Positional encoding	sinusoidal	1.6198
	RoPE	**1.6093**
Dropout	0.05	1.6093
	0.1	**1.5675**
Activation	SwiGLU	1.5675
	GELU	**1.5597**

Table 3: 24-epoch experiments used to choose the final decoder pre-training recipe.

Epoch	Clean	Light	Heavy
0 - 2	100%	0%	0%
3 - 7	40%	40%	20%
8 - end	20%	30%	50%

Table 4: The noise level sample probability for each stage in curriculum learning.

Key variables	**Operational definition**
Music Content Score	Weighted sum of pitch-class F1 (0.30), rhythm F1 (0.25), pitch-class-onset-duration F1 (0.20), pitch-class timeline DTW similarity (0.15), and rhythm timeline DTW similarity (0.10).
Notation Score	Weighted sum of pitch-onset-duration-staff F1 (0.35), strict measure accuracy (0.25), key-event F1 (0.15), time-event F1 (0.15), and XML structural similarity (0.10).
Pitch-Class-Onset-Duration F1	Note-level F1 under pitch-class, onset, and duration matching with predefined onset and duration tolerances.
Rhythm F1	Note-level F1 under onset and duration matching only, without pitch constraints.

Table 5: Main evaluation variables used in our experiments.

Overall	**MuseScore**	**Beyer**	**Our Model**	**music21**
Musical	0.7573	0.2948	0.6704	0.7452
Notation	0.5509	0.1977	0.5253	0.4860
Pitch/Onset/Duration F1	0.5180	0.0275	0.3530	0.4690
Rhythm F1	0.6934	0.0717	0.5253	0.5767

Table 6: Appendix A1: Overall performance across models.

Clean	**MuseScore**	**Beyer**	**Our Model**	**music21**
Musical	0.8354	0.2825	0.6708	0.8540
Notation	0.6455	0.1940	0.5256	0.6221
Pitch/Onset/Duration F1	0.6468	0.0293	0.3444	0.7093
Rhythm F1	0.8306	0.0695	0.5217	0.7645

Table 7: Appendix A2: Performance on clean MIDI inputs.

Light	**MuseScore**	**Beyer**	**Our Model**	**music21**
Musical	0.7503	0.3062	0.6712	0.7159
Notation	0.5607	0.2009	0.5287	0.4457
Pitch/Onset/Duration F1	0.5197	0.0286	0.3571	0.4180
Rhythm F1	0.6764	0.0784	0.5316	0.5264

Table 8: Appendix A3: Performance on lightly perturbed MIDI inputs.

Heavy	**MuseScore**	**Beyer**	**Our Model**	**music21**
Musical	0.6844	0.2943	0.6692	0.6592
Notation	0.4444	0.1976	0.5217	0.3821
Pitch/Onset/Duration F1	0.3847	0.0247	0.3567	0.2886
Rhythm F1	0.5700	0.0667	0.5220	0.3351

Table 9: Appendix A4: Performance on heavily perturbed MIDI inputs.